

3.1 — Shell Scripting for EdgeAI

Edge AI Engineering Bootcamp | Analog Data

Bash Scripting — Beginner Notes

"Automating Your Pi Like a Pro"

Before We Start — What Do You Already Need to Know?

Just these basics:

- You've opened a terminal before
- You know how to type commands like `ls`, `cd`, `pwd`
- You know what a file and folder is

That's it. No programming experience needed for this section.

1. What Is a Shell Script?

Every time you type a command in the terminal, you're talking to the **shell** — a program that reads what you type and runs it.

A **shell script** is simply a text file that contains a list of commands. Instead of typing them one by one, you write them all in a file and run the file. The shell reads each line and runs it in order — like a recipe.

Real life analogy: Imagine you make tea every morning — boil water, add tea bag, wait 3 minutes, add milk. Instead of thinking about each step every morning, you write it down as a recipe once. Shell scripts are exactly that — a recipe for your computer.

Why is this useful on a Raspberry Pi?

- Automatically start your AI model when the Pi boots
 - Download and verify a new model file every night
 - Check system health (temperature, memory, disk) and send a report
 - Do repetitive tasks without typing the same 10 commands every time
-

2. Your First Shell Script

Step 1 — Create the file

Open a terminal and type:

```
nano hello.sh
```

This opens a text editor. Type this:

```
#!/bin/bash
echo "Hello from my first script!"
echo "Today's date is:"
date
```

Save with `Ctrl + O`, press Enter, then exit with `Ctrl + X`.

Step 2 — What is `#!/bin/bash` ?

The very first line of every shell script must be:

```
#!/bin/bash
```

This is called a **shebang** (pronounced "sha-bang"). It tells the operating system: "Use the Bash shell to run this file." Without it, the system might use the wrong shell and your script may fail.

Rule: Always start every shell script with `#!/bin/bash` on line 1.

Step 3 — Make it executable

By default, a new file can't be run as a program. You need to give it permission:

```
chmod +x hello.sh
```

Step 4 — Run it

```
./hello.sh
```

Output:

```
Hello from my first script!
Today's date is:
Thu Jul 17 18:30:00 IST 2026
```

The `./` means "run this file from the current directory."

3. Variables — Storing Information

A **variable** is a named container that holds a value. You store something in it, then use it later.

Creating a variable

```
#!/bin/bash
name="Raspberry Pi"
temperature=52
```

Rules for variables:

- No spaces around the `=` sign — `name = "Pi"` is WRONG, `name="Pi"` is RIGHT
- Variable names: letters, numbers, underscores only — no spaces, no dashes
- By convention, use uppercase for important/constant values

Using a variable — add `$` before the name

```
#!/bin/bash
name="Raspberry Pi"
temperature=52

echo "Device: $name"
echo "Temperature: $temperature degrees"
echo "Device is $name and running at ${temperature}C"
```

Output:

```
Device: Raspberry Pi
Temperature: 52 degrees
Device is Raspberry Pi and running at 52C
```

The `${}` curly brace form is useful when your variable is right next to other text — it clearly shows where the variable name ends.

Storing command output in a variable

You can capture the output of a command and store it in a variable using `$(...)`:

```
#!/bin/bash
current_date=$(date)
current_user=$(whoami)
disk_space=$(df -h / | tail -1 | awk '{print $5}')

echo "Date: $current_date"
echo "User: $current_user"
echo "Disk used: $disk_space"
```

Output:

```
Date: Thu Jul 17 18:30:00 IST 2026
User: pi
Disk used: 34%
```

Read-only variables (constants)

```
readonly MODEL_PATH="/home/pi/models/model.tflite"
readonly MAX_TEMP=70
```

Using `readonly` means the variable cannot be changed later in the script.

4. Arguments — Passing Information to Your Script

When you run a script, you can pass values to it directly on the command line. These are called **arguments**.

```
./myscript.sh argument1 argument2 argument3
```

Inside the script, these are available as special variables:

Variable	What it contains
<code>\$0</code>	The name of the script itself
<code>\$1</code>	First argument
<code>\$2</code>	Second argument
<code>\$3</code>	Third argument
<code>\$#</code>	Total number of arguments passed
<code>\$@</code>	All arguments as a list

Example — a script that greets a person

```
#!/bin/bash
# save as greet.sh

name=$1
city=$2

echo "Hello, $name!"
echo "You are from $city."
echo "Script name: $0"
echo "Total arguments received: $#"
```

Run it:

```
./greet.sh Rahul Bengaluru
```

Output:

```
Hello, Rahul!
You are from Bengaluru.
```

```
Script name: ./greet.sh
Total arguments received: 2
```

Checking if an argument was provided

```
#!/bin/bash
if [ -z "$1" ]; then
    echo "Error: Please provide a name."
    echo "Usage: ./greet.sh <name>"
    exit 1
fi

echo "Hello, $1!"
```

`-z "$1"` means "if \$1 is empty." `exit 1` stops the script with an error code.

5. `if` Statement — Making Decisions

The `if` statement runs a block of code only if a condition is true.

Basic structure

```
if [ condition ]; then
    # runs if condition is true
elif [ other_condition ]; then
    # runs if other_condition is true
else
    # runs if nothing above was true
fi
```

Important: Spaces matter inside `[ ]` — always have a space after `[` and before `]`.

Comparing numbers

```
#!/bin/bash
temperature=65

if [ $temperature -gt 70 ]; then
    echo "CRITICAL: Pi is overheating!"
elif [ $temperature -gt 60 ]; then
    echo "WARNING: Temperature is high."
else
    echo "OK: Temperature is normal."
fi
```

Number comparison operators:

Operator	Meaning	Example
<code>-eq</code>	Equal to	<code>[\$a -eq \$b]</code>
<code>-ne</code>	Not equal to	<code>[\$a -ne \$b]</code>
<code>-gt</code>	Greater than	<code>[\$a -gt 70]</code>
<code>-ge</code>	Greater than or equal	<code>[\$a -ge 70]</code>
<code>-lt</code>	Less than	<code>[\$a -lt 70]</code>
<code>-le</code>	Less than or equal	<code>[\$a -le 70]</code>

Comparing text (strings)

```
#!/bin/bash
status="running"

if [ "$status" = "running" ]; then
    echo "Service is active"
elif [ "$status" = "stopped" ]; then
    echo "Service is down"
fi
```

Always put string variables in double quotes — `"$status"` not `$status`. Without quotes, if the variable is empty, the script will crash with a confusing error.

String comparison operators:

Operator	Meaning
<code>=</code>	Equal
<code>!=</code>	Not equal
<code>-z "\$var"</code>	String is empty
<code>-n "\$var"</code>	String is NOT empty

Checking if a file or folder exists

```
#!/bin/bash
model_file="/home/pi/models/model.tflite"

if [ -f "$model_file" ]; then
    echo "Model file found. Starting inference..."
else
    echo "Model file not found! Please download it first."
    exit 1
fi
```

File/folder check operators:

Operator	Meaning
-f "\$path"	File exists
-d "\$path"	Directory (folder) exists
-e "\$path"	File or directory exists
-r "\$path"	File is readable
-x "\$path"	File is executable

6. for Loop — Repeating Something a Set Number of Times

A `for` loop repeats a block of code for each item in a list.

Loop over a list of values

```
#!/bin/bash
for fruit in apple banana mango orange; do
    echo "I like $fruit"
done
```

Output:

```
I like apple
I like banana
I like mango
I like orange
```

Loop over a range of numbers

```
#!/bin/bash
for i in 1 2 3 4 5; do
    echo "Count: $i"
done

# Cleaner way using {start..end}
for i in {1..10}; do
    echo "Number: $i"
done
```

Loop over files in a folder

```
#!/bin/bash
for file in /home/pi/models/*.tflite; do
    echo "Found model: $file"
done
```

Loop over command output

```
#!/bin/bash
# Run a health check for each sensor
for sensor_id in $(cat /home/pi/sensors.txt); do
    echo "Checking sensor: $sensor_id"
    # run your check here
done
```

7. while Loop — Repeating Until a Condition Changes

A `while` loop keeps running as long as a condition is true. Use it when you don't know in advance how many times you'll need to loop.

Basic while loop

```
#!/bin/bash
count=1

while [ $count -le 5 ]; do
    echo "Loop number: $count"
    count=$((count + 1))    # increment count by 1
done
```

Output:

```
Loop number: 1
Loop number: 2
Loop number: 3
Loop number: 4
Loop number: 5
```

`=$((count + 1))` is how you do arithmetic in bash — wrap the math in `$( ( ))`.

Infinite loop — runs forever until you break it

```
#!/bin/bash
# Monitor temperature every 5 seconds forever
while true; do
    temp=$(vcgencmd measure_temp | cut -d= -f2)
    echo "Temperature: $temp"
    sleep 5
done
```

Stop an infinite loop in the terminal with `Ctrl + C`.

While loop — read a file line by line

```
#!/bin/bash
while IFS= read -r line; do
    echo "Processing: $line"
done < /home/pi/config.txt
```

8. `case` Statement — Cleaner Way to Handle Many Options

When you have many possible values to check, `case` is cleaner than writing many `elif` blocks.

Basic structure

```
case "$variable" in
    "value1")
        # do this if variable equals value1
        ;;
    "value2")
        # do this if variable equals value2
        ;;
    *)
        # default — runs if nothing above matched
        ;;
esac
```

Note: Each option ends with `;;` and the whole block ends with `esac` (case backwards).

Example — handling script arguments

```
#!/bin/bash
action=$1

case "$action" in
    "start")
        echo "Starting the inference service..."
        ;;
    "stop")
        echo "Stopping the inference service..."
        ;;
    "restart")
        echo "Restarting the inference service..."
        ;;
    "status")
        echo "Checking status..."
        ;;
    *)
        echo "Unknown action: $action"
        echo "Usage: ./service.sh start|stop|restart|status"
        exit 1
        ;;
esac
```

Run it:

```
./service.sh start      # prints: Starting the inference service...
./service.sh stop       # prints: Stopping the inference service...
./service.sh dance      # prints: Unknown action: dance
```

9. `set -euo pipefail` — Making Your Scripts Safe

This is one of the most important lines you'll ever add to a shell script. Put it at the top of every serious script, right after `#!/bin/bash`.

```
#!/bin/bash
set -euo pipefail
```

Let's break down what each part does:

`set -e` — Exit on any error

Normally, if a command in your script fails, Bash just skips it and continues to the next line. This can cause disasters — imagine a script that is supposed to download a model, verify it, and then start inference. If the download fails but the script keeps going, it will try to run inference on a corrupt or missing file.

`-e` makes the script **stop immediately** if any command fails.

```
#!/bin/bash
set -e

echo "Downloading model..."
wget https://example.com/model.tflite    # if this fails...

echo "Starting inference..."            # ...this line NEVER runs (with -e)
python3 run_inference.py                 # ...and this never runs either
```

Without `set -e`, if the download fails, the script would still try to run inference on a file that doesn't exist.

`set -u` — Error on undefined variables

Without `-u`, if you type a variable name wrong, Bash silently treats it as an empty string. This can cause very confusing bugs.

```
# Without set -u
model_path="/home/pi/models/model.tflite"
echo $model_path    # typo! but Bash just prints nothing — no error
rm -rf $model_path  # DANGER: if model_path is empty, this could delete wrong things

# With set -u
set -u
echo $model_path    # script stops immediately with: "model_path: unbound variable"
```

`-u` catches typos in variable names immediately instead of letting them cause hidden damage.

pipefail — Catch errors inside pipes

A **pipe** (`|`) connects two commands — the output of one becomes the input of the next:

```
cat log.txt | grep "ERROR" | wc -l
```

Without `pipefail`, if `cat log.txt` fails (file doesn't exist), Bash only checks the last command in the pipe (`wc -l`) — which might succeed because it just counted zero lines. The error in `cat` is silently ignored.

With `pipefail`, if **any** command in a pipe fails, the whole pipe is considered failed.

Together — the safe script header

```
#!/bin/bash
set -euo pipefail

# Now your script will:
# - Stop immediately on any error (-e)
# - Error immediately on typos in variable names (-u)
# - Catch errors in piped commands (pipefail)
```

Teaching point: `set -euo pipefail` is a professional habit. Experienced engineers add it to every script. Get students in the habit of putting it on line 2 of every script they write.

10. Practical Script 1 — System Health Reporter

This script checks the Pi's temperature, memory, and disk space, then prints a health report.

```
#!/bin/bash
set -euo pipefail

# =====
# System Health Reporter for Raspberry Pi
# Usage: ./health_report.sh
# =====

echo "=====
echo "  RASPBERRY PI HEALTH REPORT"
echo "  $(date)"
echo "=====

# --- Temperature ---
temp_raw=$(vcgencmd measure_temp)           # output: temp=52.3'C
temp=$(echo "$temp_raw" | cut -d= -f2)       # extract: 52.3'C
echo ""
echo "CPU Temperature : $temp"

# Check if overheating
```

```

temp_num=$(echo "$temp" | cut -d. -f1)          # get just the number: 52
if [ "$temp_num" -gt 80 ]; then
    echo "STATUS           : ❌ CRITICAL - Overheating!"
elif [ "$temp_num" -gt 65 ]; then
    echo "STATUS           : ⚠️  WARNING - Running hot"
else
    echo "STATUS           : ✅ OK"
fi

# --- Memory ---
echo ""
echo "Memory Usage:"
free -h | grep Mem | awk '{printf "  Total: %s | Used: %s | Free: %s\n", $2, $3, $4}'

# --- Disk Space ---
echo ""
echo "Disk Usage (root partition):"
df -h / | tail -1 | awk '{printf "  Total: %s | Used: %s | Free: %s | Used%%: %s\n", $2, $3, $4, $5}'

# Check if disk is almost full
disk_used=$(df / | tail -1 | awk '{print $5}' | cut -d% -f1)
if [ "$disk_used" -gt 90 ]; then
    echo "  STATUS: ❌ CRITICAL - Disk almost full!"
elif [ "$disk_used" -gt 75 ]; then
    echo "  STATUS: ⚠️  WARNING - Disk getting full"
else
    echo "  STATUS: ✅ OK"
fi

# --- CPU Load ---
echo ""
echo "CPU Load (1 min avg):"
load=$(cat /proc/loadavg | awk '{print $1}')
echo "  Load Average: $load"

# --- Throttling Check ---
echo ""
echo "Throttling Status:"
throttled=$(vcgencmd get_throttled)
if [ "$throttled" = "throttled=0x0" ]; then
    echo "  STATUS: ✅ No throttling detected"
else
    echo "  STATUS: ⚠️  WARNING - Throttling detected: $throttled"
fi

echo ""
echo "=====
echo "Report generated: $(date)"
echo "=====

```

Save as `health_report.sh` , make executable with `chmod +x health_report.sh` , run with `./health_report.sh` .

11. Practical Script 2 — Model Download + SHA256 Verify

When you download a model file from the internet, you need to verify it wasn't corrupted during download. The SHA256 hash is like a fingerprint — if the file is damaged or tampered with, the hash will be different.

```
#!/bin/bash
set -euo pipefail

# =====
# Model Download and Verification Script
# Usage: ./download_model.sh <url> <expected_sha256>
# Example:
#   ./download_model.sh \
#     https://example.com/model.tflite \
#     a3f9c2d1e8b74f6...
# =====

# Check arguments
if [ "$#" -ne 2 ]; then
    echo "ERROR: Wrong number of arguments."
    echo "Usage: $0 <download_url> <expected_sha256_hash>"
    exit 1
fi

DOWNLOAD_URL="$1"
EXPECTED_HASH="$2"
DOWNLOAD_DIR="/home/pi/models"
FILENAME=$(basename "$DOWNLOAD_URL") # extract filename from URL
FILEPATH="$DOWNLOAD_DIR/$FILENAME"

echo "=====
echo " Model Download & Verification Script"
echo "=====
echo "URL       : $DOWNLOAD_URL"
echo "Filename  : $FILENAME"
echo "Save to   : $FILEPATH"
echo ""

# --- Step 1: Create models directory if it doesn't exist ---
if [ ! -d "$DOWNLOAD_DIR" ]; then
    echo "Creating directory: $DOWNLOAD_DIR"
    mkdir -p "$DOWNLOAD_DIR"
fi

# --- Step 2: Download the file ---
echo "Downloading model..."
wget --show-progress -O "$FILEPATH" "$DOWNLOAD_URL"
echo "Download complete."
echo ""

# --- Step 3: Verify the SHA256 hash ---
echo "Verifying file integrity..."
```

```

ACTUAL_HASH=$(sha256sum "$FILEPATH" | awk '{print $1}')

echo "Expected hash : $EXPECTED_HASH"
echo "Actual hash   : $ACTUAL_HASH"
echo ""

if [ "$ACTUAL_HASH" = "$EXPECTED_HASH" ]; then
    echo "✅ Verification PASSED – File is intact."
    echo "Model saved to: $FILEPATH"
else
    echo "❌ Verification FAILED – File is corrupted or tampered!"
    echo "Deleting the bad file..."
    rm -f "$FILEPATH"
    exit 1
fi

echo ""
echo "Done! Model is ready to use."

```

How SHA256 verification works:

- When a model provider publishes a file, they also publish its SHA256 hash (a long string of letters and numbers)
- After you download the file, you compute its hash
- If the two hashes match → file is exactly the same as the original
- If they don't match → the file was corrupted during download or someone changed it

12. `cron` — Running Scripts Automatically on a Schedule

`cron` is the Linux task scheduler. It runs commands or scripts automatically at times you specify — every minute, every hour, every night at 2am, every Monday, etc.

Think of it like setting an alarm on your phone, but instead of waking you up, it runs a script.

Opening the crontab

Your personal schedule of tasks is stored in the **crontab** (cron table). Edit it with:

```
crontab -e
```

The first time, it asks which editor you want — choose `nano` (option 1).

The crontab format

Each line in the crontab follows this format:

```
minute hour day_of_month month day_of_week command_to_run
```

The five time fields:

Field	Allowed values	Meaning
Minute	0–59	Which minute of the hour
Hour	0–23	Which hour (24-hour format)
Day of month	1–31	Which day of the month
Month	1–12	Which month
Day of week	0–7	0 and 7 = Sunday, 1 = Monday, etc.

Use `*` to mean "every" — every minute, every hour, every day, etc.

Reading cron expressions — examples

```
# |----- minute (0–59)
# |----- hour (0–23)
# |----- day of month (1–31)
# |----- month (1–12)
# |----- day of week (0–7, 0 and 7 = Sunday)
# |-----
# * * * * * command

* * * * *      # every minute
0 * * * *      # at the start of every hour (e.g. 1:00, 2:00, 3:00...)
0 2 * * *      # every day at 2:00 AM
0 2 * * 0       # every Sunday at 2:00 AM
0 2 1 * *       # on the 1st day of every month at 2:00 AM
*/5 * * * *     # every 5 minutes
0 9–17 * * 1–5  # every hour from 9am to 5pm, Monday to Friday
```

Real crontab entries for Edge AI on Pi

```
# Run a health check every 5 minutes and save to a log file
*/5 * * * * /home/pi/scripts/health_report.sh >> /home/pi/logs/health.log 2>&1

# Download latest model every night at 2:30 AM
30 2 * * * /home/pi/scripts/download_model.sh https://example.com/model.tflite
abc123hash >> /home/pi/logs/model_update.log 2>&1

# Run batch inference every day at 6:00 AM on last night's captured data
0 6 * * * /home/pi/scripts/run_batch_inference.sh >> /home/pi/logs/inference.log
2>&1

# Clean up old log files every Sunday at 3:00 AM (keep logs from last 7 days)
0 3 * * 0 find /home/pi/logs -name "*.log" -mtime +7 -delete
```

What is `>> /path/to/log 2>&1` ?

This is output redirection — it saves everything the script prints to a log file instead of the terminal.

Symbol	Meaning
>	Write output to file (overwrites existing)
>>	Append output to file (adds to the end)
2>&1	Also capture error messages (not just normal output)

Without `>> logfile 2>&1`, any output from your cron job simply disappears — you'd never know if it worked or failed.

Viewing your current crontab

```
crontab -l    # list all scheduled tasks
```

Removing your crontab

```
crontab -r    # remove all scheduled tasks (use carefully!)
```

Important: Always use full paths in cron

When cron runs a script, it doesn't know where you are in the file system. So instead of:

```
# WRONG in cron - "scripts" relative to where?
*/5 * * * * scripts/health_report.sh
```

Always write the full path:

```
# RIGHT - cron knows exactly where to find it
*/5 * * * * /home/pi/scripts/health_report.sh
```

13. Functions in Shell Scripts

Functions let you group commands together and reuse them.

```
#!/bin/bash
set -euo pipefail

# Define a function
check_temperature() {
    local temp_raw
    temp_raw=$(vcgencmd measure_temp)
    local temp
    temp=$(echo "$temp_raw" | cut -d= -f2 | cut -d. -f1)
    echo "$temp"
}

# Define a function with arguments
```

```

log_message() {
    local level="$1"
    local message="$2"
    echo "[$(date '+%H:%M:%S')] [$level] $message"
}

# Call the functions
current_temp=$(check_temperature)
log_message "INFO" "Current temperature: ${current_temp}C"

if [ "$current_temp" -gt 70 ]; then
    log_message "WARN" "Temperature too high!"
fi

```

Important: Use `local` inside functions for variables — this keeps them from accidentally affecting the rest of the script.

14. Reading User Input

Sometimes you want the script to ask the user a question and wait for an answer:

```

#!/bin/bash

echo "Enter your name:"
read name

echo "Do you want to run inference now? (yes/no):"
read answer

if [ "$answer" = "yes" ]; then
    echo "Starting inference for $name..."
else
    echo "OK, exiting."
fi

```

Read with a prompt on the same line:

```

read -p "Enter model path: " model_path
echo "Using model: $model_path"

```

Read a password (characters hidden while typing):

```

read -s -p "Enter password: " password
echo ""
echo "Password received (not showing it)."

```

15. Colours in Scripts — Making Output Readable

For scripts that generate reports, colour-coded output is much easier to read:

```
#!/bin/bash

# Colour codes
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'      # NC = No Colour (reset)

echo -e "${GREEN}✅ Service is running${NC}"
echo -e "${RED}❌ Error: File not found${NC}"
echo -e "${YELLOW}⚠ Warning: Disk is 80% full${NC}"
echo -e "${BLUE}ℹ Info: Starting inference...${NC}"
```

Note: Use `echo -e` (not just `echo`) when you have colour codes — `-e` enables interpretation of special characters.

16. Quick Reference Card

Concept	Syntax	Example
Shebang	<code>#!/bin/bash</code>	Always line 1
Safe mode	<code>set -euo pipefail</code>	Always line 2
Variable	<code>name="value"</code>	<code>model="resnet.tflite"</code>
Use variable	<code>\$name</code> or <code>\${name}</code>	<code>echo \$model</code>
Command output	<code>var=\$(command)</code>	<code>temp=\$(vcgencmd measure_temp)</code>
Argument 1	<code>\$1</code>	<code>filename=\$1</code>
Argument count	<code>\$#</code>	<code>if [\$# -ne 2]</code>
if/then/fi	<code>if [cond]; then ... fi</code>	See Section 5
for loop	<code>for x in list; do ... done</code>	See Section 6
while loop	<code>while [cond]; do ... done</code>	See Section 7
case	<code>case "\$var" in val) ;; esac</code>	See Section 8
Function	<code>myFunc() { ... }</code>	See Section 13
Run script	<code>./script.sh</code>	After <code>chmod +x</code>
Make executable	<code>chmod +x script.sh</code>	Run once per script
Edit crontab	<code>crontab -e</code>	Opens schedule editor
List crontab	<code>crontab -l</code>	Shows all scheduled tasks
Arithmetic	<code>\$((a + b))</code>	<code>count=\$((count + 1))</code>
String empty?	<code>[-z "\$var"]</code>	True if empty

Concept	Syntax	Example
File exists?	<code>[-f "\$path"]</code>	True if file exists

17. Common Beginner Mistakes

Mistake	What Happens	Fix
Space around <code>=</code> in variable	<code>name = "Pi"</code> gives error	<code>name="Pi"</code> — no spaces
Forgetting <code>\$</code> to use variable	Prints the word, not the value	Always <code>echo \$name</code> , not <code>echo name</code>
No space inside <code>[]</code>	<code>[-z "\$var"]</code> gives error	<code>[-z "\$var"]</code> — spaces required
Missing <code>;;</code> in case	Script behaves unpredictably	End every case option with <code>;;</code>
Using <code>time.sleep</code> instead of <code>sleep</code>	<code>time.sleep</code> is Python, not Bash	Use <code>sleep 5</code> in Bash
Not using full path in cron	Task silently fails	Always write <code>/home/pi/scripts/script.sh</code>
No <code>2>&1</code> in cron	Errors never recorded	Always add <code>>> log.txt 2>&1</code>
Forgetting <code>chmod +x</code>	"Permission denied" error	Run <code>chmod +x script.sh</code> once

Summary Checklist for Students

- ☐ I can create a shell script file with the correct shebang line
- ☐ I know what `set -euo pipefail` does and why I should always use it
- ☐ I can create variables and use them with `$`
- ☐ I can capture command output into a variable using `$( )`
- ☐ I can access script arguments using `$1`, `$2`, `$#`
- ☐ I can write an `if` statement to compare numbers and strings
- ☐ I can write a `for` loop over a list, a range, and files in a folder
- ☐ I can write a `while` loop that runs until a condition changes
- ☐ I can write a `case` statement to handle multiple options cleanly
- ☐ I can write and read the system health reporter script
- ☐ I can explain what SHA256 verification does and why it matters
- ☐ I can edit the crontab and schedule a script to run at a specific time
- ☐ I can write the crontab entry to run a model update every night at 2:30 AM

☐ I know why full paths and log redirection are essential in cron jobs